Guess X - SECURITY_ASSESSMENT

- 1 Application Security Review & Vulnerability Assessment (ASVA)
  - 1.1 Guess X — Lowest Unique Bid Auction Platform
  - 1.2 Document Control
    - 1.2.1 Version History
  - 1.3 Table of Contents
  - 1.4 1. Executive Summary & Scope
    - 1.4.1 1.1 Overview
    - 1.4.2 1.2 Testing Methodology
  - 1.5 2. Injection & Common Web Vulnerabilities
    - 1.5.1 2.1 Cross-Site Scripting (XSS)
    - 1.5.2 2.2 XML Injection (XXE)
    - 1.5.3 2.3 SQL Injection (SQLi)
    - 1.5.4 2.4 Command Injection
    - 1.5.5 2.5 Path Traversal
    - 1.5.6 2.6 Server-Side Request Forgery (SSRF)
    - 1.5.7 2.7 Insecure Direct Object Reference (IDOR)
  - 1.6 3. Authentication & Access Control
    - 1.6.1 3.1 Improper Authentication
    - 1.6.2 3.2 Authentication Bypass Using Alternate Paths
    - 1.6.3 3.3 Missing Authentication for Critical Functions
    - 1.6.4 3.4 Authorization Bypass Through SQL Key Test / Parameter Tampering
    - 1.6.5 3.5 Missing Authorization Test (Broken Function Level Authorization)
    - 1.6.6 3.6 Improper Privilege Management
  - 1.7 4. Passwords, Credentials & Cryptography
    - 1.7.1 4.1 Hardcoded Passwords & Secrets
    - 1.7.2 4.2 Weak Password Requirements
    - 1.7.3 4.3 Insufficiently Protected Credentials
    - 1.7.4 4.4 Password Hashing with Insufficient Work Factor
    - 1.7.5 4.5 Key Exchange Without Entity Authentication
    - 1.7.6 4.6 Broken or Risky Cryptographic Algorithms
    - 1.7.7 4.7 Certificate Chain of Trust Verification
    - 1.7.8 4.8 Cleartext Transmission of Sensitive Information
  - 1.8 5. Data Handling, Integrity & State Management
    - 1.8.1 5.1 Remote File Inclusion (RFI)
    - 1.8.2 5.2 Unrestricted File Upload

        - 1.8.3 5.3 External Control of Critical State Data
        - 1.8.4 5.4 Sensitive Query Strings in GET Requests
        - 1.8.5 5.5 Insufficient Verification of Data Authenticity
        - 1.8.6 5.6 Missing Support for Integrity Checks
        - 1.8.7 5.7 Insecure Storage of Sensitive Information
        - 1.8.8 5.8 Omission of Security-Relevant Information in Logs
    - 1.9 6. Cookies, Sessions & Redirects
        - 1.9.1 6.1 Cross-Site Request Forgery (CSRF)
        - 1.9.2 6.2 Reliance on Cookies Without Validation
        - 1.9.3 6.3 Sensitive Cookies Without HTTPOnly Flag
        - 1.9.4 6.4 Sensitive Cookies Without Secure Attribute
        - 1.9.5 6.5 Open Redirects
        - 1.9.6 6.6 Permissive Cross-Origin Resource Sharing (CORS)
    - 1.10 7. Configuration, Errors & System Environment
        - 1.10.1 7.1 Race Conditions & Concurrency
        - 1.10.2 7.2 Metadata Manipulation & Clock Skew
        - 1.10.3 7.3 Log Injection (Improper Neutralization for Logs)
        - 1.10.4 7.4 Exposure of Sensitive Information in Error Messages
        - 1.10.5 7.5 Uncaught Exceptions & Service Stability
        - 1.10.6 7.6 Components with Known Vulnerabilities
    - 1.11 8. Business Logic Security & Abuse Testing
        - 1.11.1 8.1 Bid / Amount Tampering
        - 1.11.2 8.2 Auction Timing Abuse
        - 1.11.3 8.3 Winner Determination Manipulation
        - 1.11.4 8.4 Payment Amount Tampering
        - 1.11.5 8.5 Wallet Double-Spend / Negative Balance
        - 1.11.6 8.6 Fee / Charge Bypass
        - 1.11.7 8.7 Workflow & State Machine Bypass
        - 1.11.8 8.8 Business Limit Bypass (Max Bids per User)
        - 1.11.9 8.9 Mass Assignment / Over-Posting
        - 1.11.10 8.10 Transaction Replay Attacks
        - 1.11.11 8.11 Currency & Rounding Manipulation
        - 1.11.12 8.12 Privileged Business Function Abuse (Administrator Bidding)
    - 1.12 9. Automated Penetration Test Verification
    - 1.13 10. Remediation & Hardening Roadmap

1 Application Security Review & Vulnerability Assessment (ASVA)

1.1 Guess X — Lowest Unique Bid Auction Platform

1.2 Document Control

Property	Value
Document Title	Application Security Review & Vulnerability Assessment (ASVA)
Target System	Guess X Platform (Microservices, Web, Mobile)
Document Version	1.0
Document Date	2026-10-19
Document Status	Approved for Review & Compliance Verification
Compliance Standards	OWASP Top 10 (2021), OWASP ASVS v4.0.3, WSTG v4.2, Ethiopian Cyber & Banking Regulations
Classification	Confidential — Internal Security & External Audit Assessment
Author	Senior Application Security & Penetration Testing Team

1.2.1 Version History

Version	Date	Author	Reviewer	Approver	Summary of Changes
1.0	2026-10-19	Security Engineering	Architecture Review Board	Chief Technology Officer	Comprehensive security review and vulnerability assessment against enterprise

Version	Date	Author	Reviewer	Approver	Summary of Changes
					checklist (Injection, Auth, Cryptography, Data Integrity, Sessions, Environment, and Business Logic).

1.3 Table of Contents

1. Executive Summary & Scope
2. Injection & Common Web Vulnerabilities
3. Authentication & Access Control
4. Passwords, Credentials & Cryptography
5. Data Handling, Integrity & State Management
6. Cookies, Sessions & Redirects
7. Configuration, Errors & System Environment
8. Business Logic Security & Abuse Testing
9. Automated Penetration Test Verification
10. Remediation & Hardening Roadmap

1.4 1. Executive Summary & Scope

1.4.1 1.1 Overview

This document delivers an end-to-end security review, threat model analysis, and vulnerability assessment for the **Guess X** reverse auction platform. Operating on a Lowest Unique Bid (LUB) model integrated with mobile banking and digital wallets, the platform handles monetary transactions, competitive bidding frequencies, and automated settlement workflows.

The assessment covers the full architectural surface: - **Identity & Wallet Service (Port 3001)**: User lifecycle, authentication, wallet ledgers, and security audit logs. - **Auction**

Engine Service (Port 3002): Real-time bidding engine, WebSocket broadcasting, Redis sorted-set scoring, and closure state machine. - **Query & Read Service (Port 3003)**: Read-optimized auction discovery, aggregated statistics, and administrative reporting. - **Frontend Clients**: React 18 Web application and React Native / Expo Mobile application. - **Data Stores**: Distributed PostgreSQL 15 (Citus) and in-memory Redis 7.

1.4.2 1.2 Testing Methodology

Testing combines: 1. **Static Code Analysis (SAST)**: Comprehensive manual and automated code audits of TypeScript controllers, services, guards, and DTOs. 2. **Dynamic Penetration Testing (DAST)**: Simulated client manipulation, request proxying, concurrent race tests, and fuzzing. 3. **Automated Penetration Test Suites**: Validation via continuous regression scripts (`scripts/e2e-security-flow.js` and `scripts/concurrency-security-stress.js`).

1.5 2. Injection & Common Web Vulnerabilities

1.5.1 2.1 Cross-Site Scripting (XSS)

- **Risk Profile**: High. Execution of attacker-supplied JavaScript within an authenticated victim's session enables token harvesting, DOM defacement, or involuntary bid placement.
- **Architecture & Defenses**:
  - React 18 automatically applies contextual output encoding to dynamic bindings within JSX expressions.
  - Raw HTML injection points (`dangerouslySetInnerHTML`) are strictly prohibited for user-supplied data across the web and mobile codebases.
  - User profiles, dispute tickets, and product descriptions are rendered as sanitized text or schema-validated strings.
- **Testing Methodology**:
  - Inject polyglot XSS strings (`<script>alert(1)</script>` , `"><img src=x onerror=alert(1)>` , `javascript:/*--></title></style>`) into registration fields (`full_name`), auction search queries, support dispute attachments, and administrative notes.
  - Verify that inputs render as encoded HTML entities in both DOM views and modal dialogues.
- **Actionable Remediation**:
  - Enforce Content Security Policy (CSP) headers at the gateway/reverse proxy:

```
Content-Security-Policy: default-src 'self'; script-src 'self'; object-s
```

1.5.2 2.2 XML Injection (XXE)

- **Risk Profile**: High. Exploitation of XML external entity parsers can lead to local file disclosure (`/etc/passwd`), SSRF, or server memory exhaustion.
- **Architecture & Defenses**:
  - All public microservice endpoints accept and produce JSON payloads exclusively (`application/json`).
  - Native XML parsing libraries (e.g., `libxmljs` , `DOMParser`) are not integrated into the runtime request pipeline.
- **Testing Methodology**:
  - Submit crafted XML payloads containing external DTD references against all POST/PUT/PATCH endpoints with `Content-Type: application/xml` .
  - Confirm the server rejects the request with HTTP 415 (Unsupported Media Type) or ignores XML entities without parsing.
- **Actionable Remediation**:
  - Enforce strict gateway-level content-type validation rejecting non-JSON payloads on REST routes.

1.5.3 2.3 SQL Injection (SQLi)

- **Risk Profile**: Critical. Tampering with database queries can allow data extraction, authentication bypass, auction outcome falsification, or ledger manipulation.
- **Architecture & Defenses**:
  - TypeORM is configured with parameterized query builders and repository methods throughout all services.
  - Direct string concatenation in raw SQL queries is strictly banned.
  - Complex aggregate operations (such as `findEarliestBidders` and settlement rollups) use parameterized placeholders (`$1, $2`).
- **Testing Methodology**:
  - Fuzz all query string parameters (`search` , `category` , `offset` , `limit`) and JSON body fields with time-based blind (`' OR SLEEP(5)--`), boolean-based (`' OR '1'='1`), and UNION-based payloads.
  - Monitor database query logs and response timings to verify query plan isolation.
- **Actionable Remediation**:
  - Grant runtime database roles minimal DML privileges (`SELECT, INSERT, UPDATE, DELETE`) with no DDL execution rights.

1.5.4 2.4 Command Injection

- **Risk Profile**: Critical. Executing arbitrary host commands could lead to host takeover or lateral movement within the cluster.
- **Architecture & Defenses**:
  - Business operations execute purely within Node.js runtime APIs. No endpoints invoke shell commands, `child_process.exec`, or `eval()`.
- **Testing Methodology**:
  - Inject shell metacharacters (`;`, `|`, `&&`, `` ` ``, `$(whoami)`) into file export names, user profile inputs, and external URLs.
- **Actionable Remediation**:
  - Prohibit the use of `child_process` invocations in production application code.

1.5.5 2.5 Path Traversal

- **Risk Profile**: High. Reading or overwriting files outside the web root could expose `.env` credentials, private keys, or system configurations.
- **Architecture & Defenses**:
  - Dynamic file paths are not exposed to client-controlled input. Product images are served via cloud object storage URLs, not local disk paths.
- **Testing Methodology**:
  - Submit traversal sequences (`../../`, `..%2f`, `%2e%2e%2f`) against export filenames, dispute evidence parameters, and asset URLs.
- **Actionable Remediation**:
  - If local file reading is ever required, use `path.resolve()` and verify canonical path containment:

```
if (!resolvedPath.startsWith(ALLOWED_BASE_DIRECTORY)) {
  throw new ForbiddenException("Invalid path traversal attempt");
}
```

1.5.6 2.6 Server-Side Request Forgery (SSRF)

- **Risk Profile**: High. Inducing backend servers to issue HTTP requests to internal services, cloud metadata APIs (`169.254.169.254`), or database interfaces.
- **Architecture & Defenses**:
  - Enforced in `ImageService`:
    - Protocol whitelisting: requires `https://`.
    - IP range filtering: resolves target DNS and blocks loopback (`127.0.0.0/8`), RFC 1918 private networks (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`), and link-local (`169.254.0.0/16`).

    - MIME validation: verifies image `Content-Type` headers and enforces a 15-second socket timeout.
- **Testing Methodology**:
    - Submit product image URLs targeting `http://127.0.0.1:6379`, `http://169.254.169.254/latest/meta-data/`, and DNS rebinding hosts.
- **Actionable Remediation**:
    - Continue resolving DNS addresses prior to connection and validate the resolved IP before socket opening.

1.5.7 2.7 Insecure Direct Object Reference (IDOR)

- **Risk Profile**: High. Unauthorized access to another user's bids, financial transactions, or wallet balance by tampering with record IDs.
- **Architecture & Defenses**:
    - Customer endpoints (`/wallet/balance`, `/wallet/transactions`, `/auctions/:id/my-bids`) resolve target data using the authenticated `req.user.id` extracted from validated JWT claims.
    - Client-supplied user IDs in request bodies are ignored on user-facing endpoints.
- **Testing Methodology**:
    - Authenticate as User A and issue requests to fetch or modify User B's bids, wallet balance, or notifications by substituting UUID parameters.
- **Actionable Remediation**:
    - Enforce server-side ownership checks across all entity lookups:

```
const bid = await this.bidRepo.findOne({ where: { id: bidId, user_id:
        req.user.id } });
if (!bid) throw new NotFoundException();
```

1.6 3. Authentication & Access Control

1.6.1 3.1 Improper Authentication

- **Risk Profile**: Critical. Allowing unauthenticated actors to access bidding, wallet debits, or administrative functions.
- **Architecture & Defenses**:
    - Core routes are protected by `JwtAuthGuard`.
    - Fail-closed design: if authentication configuration is missing or token verification errors occur, requests are rejected with HTTP 401 (`Authentication service unavailable`).

- **Testing Methodology**:
  - Submit requests with missing, malformed, expired, or unsigned tokens against all protected endpoints.
- **Actionable Remediation**:
  - Maintain short-lived access tokens (15 minutes) combined with rolling refresh token rotation.

1.6.2 3.2 Authentication Bypass Using Alternate Paths

- **Risk Profile**: High. Bypassing authentication filters through URL casing, trailing slashes, or proxy header manipulation.
- **Architecture & Defenses**:
  - Route guards are bound via NestJS controller and method decorators (`@UseGuards(JwtAuthGuard)`), ensuring execution regardless of URL encoding or path normalization.
- **Testing Methodology**:
  - Test alternate URL structures (`/api/v1/%61uth/profile`, `/api/v1/wallet/balance/`, `X-Rewrite-URL: /admin`).
- **Actionable Remediation**:
  - Normalize incoming request paths at the reverse proxy prior to upstream dispatch.

1.6.3 3.3 Missing Authentication for Critical Functions

- **Risk Profile**: Critical. Administrative functions (auction force-close, manual winner declaration, user bans) callable anonymously.
- **Architecture & Defenses**:
  - All administrative controllers (`/api/v1/admin/*`) require `JwtAuthGuard` and `RolesGuard(['admin'])`.
- **Testing Methodology**:
  - Directly invoke `POST /api/v1/admin/auctions/:id/close` with no headers or an empty Authorization header.
- **Actionable Remediation**:
  - Maintain automated regression tests verifying that every administrative route returns HTTP 401 without valid credentials.

1.6.4 3.4 Authorization Bypass Through SQL Key Test / Parameter Tampering

- **Risk Profile**: High. Elevating privileges by injecting `role: 'admin'` or `is_admin: true` in user self-service requests.
- **Architecture & Defenses**:

    - Global `ValidationPipe` with `whitelist: true` and `forbidNonWhitelisted: true` rejects requests containing unmapped properties.
- **Testing Methodology**:
    - Send `POST /auth/register` and `PATCH /auth/profile` containing elevated role attributes.
- **Actionable Remediation**:
    - Keep role and permission assignment logic isolated strictly to dedicated administrative endpoints.

1.6.5 3.5 Missing Authorization Test (Broken Function Level Authorization)

- **Risk Profile**: Critical. Low-privilege users invoking administrative or internal service endpoints.
- **Architecture & Defenses**:
    - `RolesGuard` strictly verifies `req.user.role === 'admin'`.
    - Service-to-service endpoints enforce `InternalAuthGuard` with `x-internal-api-key`.
- **Testing Methodology**:
    - Issue requests to `/api/v1/admin/auctions` using a regular customer JWT.
- **Actionable Remediation**:
    - Unit and integration tests must assert HTTP 403 Forbidden for customer tokens targeting administrative routes.

1.6.6 3.6 Improper Privilege Management

- **Risk Profile**: High. Retaining elevated privileges after administrative demotion or account suspension.
- **Architecture & Defenses**:
    - User ban and role statuses are validated against database records and a Redis blacklist (`guessx:banned-users`) on every authenticated request.
- **Testing Methodology**:
    - Demote or ban an active administrative user; verify immediate rejection on subsequent requests.
- **Actionable Remediation**:
    - Maintain real-time Redis token blacklisting for revoked sessions and role adjustments.

1.7 4. Passwords, Credentials & Cryptography

1.7.1 4.1 Hardcoded Passwords & Secrets

- **Risk Profile**: Critical. Exposure of signing keys or database credentials in source control.
- **Architecture & Defenses**:
  - All secrets (`JWT_SECRET`, `INTERNAL_API_KEY`, `DATABASE_URL`) are loaded from environment variables.
  - Startup guards crash the service if default/insecure keys (`guessx-jwt-secret`, `default-secret-change-me`) are detected.
- **Testing Methodology**:
  - Scan repository history using automated secret scanners (`gitleaks`, `trufflehog`).
- **Actionable Remediation**:
  - Use pre-commit hooks and CI checks to prevent accidental commits containing credentials.

1.7.2 4.2 Weak Password Requirements

- **Risk Profile**: Medium. Users choosing easily guessable passwords susceptible to dictionary attacks.
- **Architecture & Defenses**:
  - Password lengths are validated via DTO constraints.
  - Rate-limiting (5 attempts per minute per identifier) prevents automated brute-forcing.
- **Testing Methodology**:
  - Attempt registering with trivial passwords (`123456`, `password`).
- **Actionable Remediation**:
  - Enforce password complexity policies: minimum 8 characters, requiring uppercase, lowercase, numbers, and symbols.

1.7.3 4.3 Insufficiently Protected Credentials

- **Risk Profile**: High. Leaking plaintext passwords, wallet PINs, or auth tokens into server logs or error traces.
- **Architecture & Defenses**:
  - Passwords and PINs are excluded from logging formatters.
  - Phone numbers are masked in audit logs (`091*****99`).
- **Testing Methodology**:
  - Generate intentional authentication failures and inspect backend console logs for credential leakage.
- **Actionable Remediation**:

- Apply global log masking filters stripping sensitive keys (`password` , `pin` , `token` , `secret`).

1.7.4 4.4 Password Hashing with Insufficient Work Factor

- **Risk Profile**: High. Fast offline cracking of leaked database hashes.
- **Architecture & Defenses**:
  - Customer passwords hashed with `bcrypt` at cost factor 12.
  - Wallet PINs hashed with `bcrypt` at cost factor 10.
- **Testing Methodology**:
  - Review database records to verify bcrypt algorithm identifiers (`$2b$12$` , `$2b$10$`).
- **Actionable Remediation**:
  - Keep bcrypt cost factors calibrated so verification requires approximately 100-250ms per check.

1.7.5 4.5 Key Exchange Without Entity Authentication

- **Risk Profile**: High. Man-in-the-Middle (MitM) credential interception during transmission.
- **Architecture & Defenses**:
  - Production deployments require TLS/HTTPS with authenticated server certificates.
- **Testing Methodology**:
  - Attempt MitM interception using an untrusted CA certificate.
- **Actionable Remediation**:
  - Enforce mutual TLS (mTLS) for container-to-container traffic in production environments.

1.7.6 4.6 Broken or Risky Cryptographic Algorithms

- **Risk Profile**: High. Weak cryptographic algorithms allowing cipher reversal.
- **Architecture & Defenses**:
  - Historical bid amounts encrypted with AES-256-GCM (`BidEncryptionService`).
  - Webhooks authenticated using HMAC-SHA256. Obsolete algorithms (MD5, SHA-1, DES) are completely avoided.
- **Testing Methodology**:
  - Inspect cryptographic function calls across all backend services.
- **Actionable Remediation**:
  - Always use authenticated encryption modes (AES-GCM, ChaCha20-Poly1305) with unique initialization vectors (IVs).

1.7.7 4.7 Certificate Chain of Trust Verification

- **Risk Profile**: High. Acceptance of invalid certificates facilitating interception.
- **Architecture & Defenses**:
  - Outbound HTTP clients enforce strict SSL certificate validation (`rejectUnauthorized: true`).
- **Testing Methodology**:
  - Point payment integrations to mock endpoints with invalid SSL certificates; verify requests are aborted.
- **Actionable Remediation**:
  - Strictly prohibit `NODE_TLS_REJECT_UNAUTHORIZED=0` across all production configurations.

1.7.8 4.8 Cleartext Transmission of Sensitive Information

- **Risk Profile**: Critical. Transmitting credentials or financial requests over unencrypted channels.
- **Architecture & Defenses**:
  - Reverse proxies enforce HTTPS redirection and HSTS headers.
- **Testing Methodology**:
  - Attempt unencrypted HTTP connections; confirm mandatory 301 redirects to HTTPS.
- **Actionable Remediation**:
  - Set `Strict-Transport-Security: max-age=31536000; includeSubDomains; preload` on all web gateways.

1.8 5. Data Handling, Integrity & State Management

1.8.1 5.1 Remote File Inclusion (RFI)

- **Risk Profile**: Critical. Remote code execution via dynamic file inclusion.
- **Architecture & Defenses**:
  - Not applicable to the Node.js/TypeScript architecture (dynamic inclusion patterns like PHP `include` are not used).

1.8.2 5.2 Unrestricted File Upload

- **Risk Profile**: High. Uploading malicious binaries or server-side scripts disguised as product photos.
- **Architecture & Defenses**:

    - File upload handlers enforce strict extension and MIME-type validation. Direct script execution is disabled on media storage.
- **Testing Methodology**:
    - Attempt uploading executable or script files (`.exe` , `.sh` , `.php` , `.svg` with embedded scripts).
- **Actionable Remediation**:
    - Store uploaded assets in cloud object storage buckets (S3/GCS) configured with `Content-Disposition: attachment` for untrusted files.

1.8.3 5.3 External Control of Critical State Data

- **Risk Profile**: Critical. Client manipulation of auction state, payment statuses, or winning prices.
- **Architecture & Defenses**:
    - Critical state transitions (auction closure, winner determination, payment settlement) are evaluated and executed exclusively on backend services.
- **Testing Methodology**:
    - Inject `"status": "CLOSED"` or `"payment_status": "PAID"` in client bid or payment payloads.
- **Actionable Remediation**:
    - Enforce strict DTO whitelisting and ignore any client-supplied status fields.

1.8.4 5.4 Sensitive Query Strings in GET Requests

- **Risk Profile**: Medium. Tokens or credentials leaking into browser history, web caches, and access logs.
- **Architecture & Defenses**:
    - Sensitive operations (login, deposit, fee deduction, bid placement) use HTTP POST with encrypted JSON request bodies.
- **Testing Methodology**:
    - Review network access logs to ensure no tokens or passwords appear in query parameters.
- **Actionable Remediation**:
    - Prohibit passing sensitive tokens or credentials in URL query strings.

1.8.5 5.5 Insufficient Verification of Data Authenticity

- **Risk Profile**: High. Accepting forged webhook events to mark unpaid auctions as paid.
- **Architecture & Defenses**:
    - Webhooks enforce HMAC-SHA256 signature verification over raw request payloads with a 300-second timestamp tolerance window.

- **Testing Methodology**:
  - Send synthetic webhook notifications with missing or forged signatures; confirm immediate HTTP 400/401 rejection.
- **Actionable Remediation**:
  - Use `crypto.timingSafeEqual` with length guards to prevent timing analysis during signature verification.

1.8.6 5.6 Missing Support for Integrity Checks

- **Risk Profile**: Medium. Silent tampering with transactional or balance records.
- **Architecture & Defenses**:
  - Relational integrity constraints, foreign keys, and unique reference IDs (`reference_id`, `transaction_id`) prevent duplicate or orphaned ledger entries.
- **Testing Methodology**:
  - Attempt inserting duplicate transaction references to test database-level uniqueness enforcement.
- **Actionable Remediation**:
  - Apply cryptographic hash chaining on critical financial audit logs.

1.8.7 5.7 Insecure Storage of Sensitive Information

- **Risk Profile**: High. Plaintext bid or credential exposure in database backups.
- **Architecture & Defenses**:
  - Passwords and PINs are stored exclusively as bcrypt hashes. Closed auction bid amounts are encrypted via AES-256-GCM.
- **Testing Methodology**:
  - Inspect database backups to verify the absence of plaintext passwords or sensitive raw bids.
- **Actionable Remediation**:
  - Enable Transparent Data Encryption (TDE) on PostgreSQL storage volumes.

1.8.8 5.8 Omission of Security-Relevant Information in Logs

- **Risk Profile**: Medium. Inability to trace security incidents or financial anomalies.
- **Architecture & Defenses**:
  - Dedicated audit logging records actor ID, IP address, action type, entity ID, and timestamps for all administrative actions, failed logins, winner rotations, and closures.
- **Testing Methodology**:
  - Trigger security events and inspect `GET /api/v1/admin/audit/logs` to confirm complete metadata capture.

- **Actionable Remediation**:
  - Forward audit logs to centralized, write-once-read-many (WORM) log storage.

1.9 6. Cookies, Sessions & Redirects

1.9.1 6.1 Cross-Site Request Forgery (CSRF)

- **Risk Profile**: Medium. Forcing authenticated browser sessions to perform unwanted state changes.
- **Architecture & Defenses**:
  - API architecture is stateless, using Bearer tokens in `Authorization` headers (immune to standard browser cookie-based CSRF).
  - For cookie-based sessions, an HMAC-SHA256 double-submit cookie and header pattern is enforced.
- **Testing Methodology**:
  - Submit cross-origin POST requests from an external site without the Authorization header.
- **Actionable Remediation**:
  - Store tokens in client storage and send via Authorization headers, or set `SameSite=Strict` on session cookies.

1.9.2 6.2 Reliance on Cookies Without Validation

- **Risk Profile**: High. Trusting client-manipulated cookie values for user identity.
- **Architecture & Defenses**:
  - Identity is established strictly via signed JWT token verification; cookies are not trusted for unverified state data.
- **Testing Methodology**:
  - Modify cookie values and verify that server identity and authorization remain unchanged.
- **Actionable Remediation**:
  - Verify cryptographic signatures on any session cookies before processing.

1.9.3 6.3 Sensitive Cookies Without HTTPOnly Flag

- **Risk Profile**: High. Session token extraction via Cross-Site Scripting (XSS).
- **Architecture & Defenses**:
  - Session and refresh cookies are issued with the `httpOnly: true` attribute.
- **Testing Methodology**:
  - Inspect `Set-Cookie` response headers to verify the `HttpOnly` flag is present.

- **Actionable Remediation**:
  - Enforce `httpOnly: true` on all authentication and session cookies.

1.9.4 6.4 Sensitive Cookies Without Secure Attribute

- **Risk Profile**: High. Cookie transmission over unencrypted HTTP connections.
- **Architecture & Defenses**:
  - Configured with `secure: true` in production environments.
- **Testing Methodology**:
  - Inspect `Set-Cookie` headers in production responses to confirm the `Secure` flag.
- **Actionable Remediation**:
  - Enforce `secure: process.env.NODE_ENV === 'production'` across all cookie-setting handlers.

1.9.5 6.5 Open Redirects

- **Risk Profile**: Medium. Redirecting users from trusted domains to phishing sites following login or payment.
- **Architecture & Defenses**:
  - Redirect destinations in payment flows are strictly restricted to predefined gateway URLs configured in environment variables.
- **Testing Methodology**:
  - Pass arbitrary URLs (`https://attacker.com` , `//attacker.com`) into redirect query parameters.
- **Actionable Remediation**:
  - Enforce server-side URL whitelisting; disallow arbitrary user-supplied redirect targets.

1.9.6 6.6 Permissive Cross-Origin Resource Sharing (CORS)

- **Risk Profile**: High. Unauthorized origins reading sensitive customer data or interacting with authenticated APIs.
- **Architecture & Defenses**:
  - CORS is restricted to trusted domains defined in `CORS_ORIGINS` . Wildcard (`*`) origins are prohibited in production.
- **Testing Methodology**:
  - Issue requests with `Origin: http://untrusted-domain.com` and verify the server does not return `Access-Control-Allow-Origin: *` .
- **Actionable Remediation**:
  - Maintain strict origin whitelisting on both REST APIs and WebSocket handshakes.

1.10 7. Configuration, Errors & System Environment

1.10.1 7.1 Race Conditions & Concurrency

- **Risk Profile**: High. Simultaneous requests enabling wallet double-spending or exceeding bid limits.
- **Architecture & Defenses**:
  - Validated via `scripts/concurrency-security-stress.js`:
    - Distributed Redis locks (`guessx:bid-fee:${auctionId}:${userId}:lock`) with 10-second TTLs prevent parallel fee deductions.
    - Pessimistic write locking on database balances.
    - Atomic SQL decrement queries (`UPDATE users SET wallet_balance = wallet_balance - :amount WHERE id = :id AND wallet_balance >= :amount`).
- **Testing Methodology**:
  - Fire 25 concurrent wallet deduction and bid requests simultaneously.
- **Actionable Remediation**:
  - Combine distributed locking with database-level `CHECK (wallet_balance >= 0)` constraints.

1.10.2 7.2 Metadata Manipulation & Clock Skew

- **Risk Profile**: Medium. Smuggling manipulated timestamps to bypass replay or expiry checks.
- **Architecture & Defenses**:
  - `NonceGuard` enforces maximum 30-second clock skew between client timestamp headers and authoritative server clock.
- **Testing Methodology**:
  - Send requests with timestamps skewed into the past (>30s) or future; verify immediate rejection.
- **Actionable Remediation**:
  - Synchronize all servers via Network Time Protocol (NTP).

1.10.3 7.3 Log Injection (Improper Neutralization for Logs)

- **Risk Profile**: Medium. Injecting CRLF sequences (`\r\n`) to forge log entries.
- **Architecture & Defenses**:
  - Structured JSON formatters encode dynamic parameters as escaped JSON strings, neutralizing CRLF injection.
- **Testing Methodology**:
  - Submit inputs containing `%0d%0a[WARN] User elevated to admin` in profile fields.

- **Actionable Remediation**:
  - Use structured logging libraries (Winston, Pino) that encode dynamic inputs safely.

1.10.4 7.4 Exposure of Sensitive Information in Error Messages

- **Risk Profile**: Medium. Leaking stack traces, file paths, or SQL queries in API error responses.
- **Architecture & Defenses**:
  - Managed by `GlobalExceptionFilter`:
    - Internal errors (5xx) return sanitized messages: `"Internal server error"`.
    - Stack traces are logged server-side with unique `requestId` identifiers and never sent to clients.
- **Testing Methodology**:
  - Trigger deliberate database constraint failures; confirm no SQL syntax or stack traces leak to the client.
- **Actionable Remediation**:
  - Ensure global exception filters intercept all unhandled exceptions across all services.

1.10.5 7.5 Uncaught Exceptions & Service Stability

- **Risk Profile**: High. Malformed payloads crashing the Node.js process, leading to Denial of Service.
- **Architecture & Defenses**:
  - Global exception filters wrap all HTTP and WebSocket handlers. Docker / PM2 supervisors ensure automatic process recovery.
- **Testing Methodology**:
  - Submit malformed JSON, cyclic structures, and null byte characters across endpoints.
- **Actionable Remediation**:
  - Run services behind process supervisors that restart crashed instances automatically.

1.10.6 7.6 Components with Known Vulnerabilities

- **Risk Profile**: High. Exploitation of CVEs in third-party dependencies.
- **Architecture & Defenses**:
  - Dependencies are regularly audited via `npm audit` across all microservices and client apps.
- **Testing Methodology**:
  - Execute `npm audit` across all 5 workspace projects.

- **Actionable Remediation**:
  - Automate dependency vulnerability scanning in CI/CD build pipelines.

1.11 8. Business Logic Security & Abuse Testing

1.11.1 8.1 Bid / Amount Tampering

- **Risk Profile**: Critical. Submitting sub-minimum, negative, or zero bid amounts to game auction costs.
- **Codebase Verification & Defenses**:
  - Enforced in `BidDto` using class-validator:
    - `@IsNumber()`, `@Min(1.00)`, `@Max(9999999999.99)`, and decimal precision capped at 2.
  - Participation fees are verified server-side against auction configuration and bounded by `@Min(1.00)` and `@Max(10000)` in `DeductFeeDto`.
- **Testing Methodology**:
  - Intercept `POST /auctions/:id/bid` and submit: `amount: -5.00`, `amount: 0.00`, `amount: 0.50`, `amount: 10000000000`.
  - Confirm all are rejected with HTTP 400 Bad Request.
- **Actionable Remediation**:
  - Re-derive all financial amounts server-side; reject client-specified fee amounts.

1.11.2 8.2 Auction Timing Abuse

- **Risk Profile**: High. Submitting bids after auction closure or gaming auto-extension triggers.
- **Codebase Verification & Defenses**:
  - Enforced via `BiddingWindowInterceptor`:
    - Validates that server clock time `now < auction.end_time`.
    - Bids submitted even 1 millisecond post-expiration are rejected with HTTP 400.
    - Extensions execute exclusively via server-side scheduler (`AuctionClosureService`).
- **Testing Methodology**:
  - Submit bids after `end_time` using modified client timestamps.
- **Actionable Remediation**:
  - Strictly enforce authoritative server time for all timing checks.

1.11.3 8.3 Winner Determination Manipulation

- **Risk Profile**: Critical. Manipulating the lowest unique bid calculation to award an unauthorized user.
- **Codebase Verification & Defenses**:
  - Redis sorted sets (`frequencies` , `unique_bids`) are accessible only to internal services.
  - `WinnerService.calculateWinners` resolves winners server-side, using single batch queries (`findEarliestBidders`) to determine authentic winning bidders.
- **Testing Methodology**:
  - Submit duplicate bid values from multiple accounts and verify that frequency increments evict the value from `unique_bids` .
- **Actionable Remediation**:
  - Keep Redis instances strictly within private VPC networks with password authentication.

1.11.4 8.4 Payment Amount Tampering

- **Risk Profile**: Critical. Paying an amount lower than the winning bid price.
- **Codebase Verification & Defenses**:
  - `PaymentService.createPaymentLink` resolves payment amounts directly from the database record (`winning_bid_amount`). Client-supplied amounts are ignored.
- **Testing Methodology**:
  - Intercept `POST /payments/:auctionId/link` and inject `"amount": 1.00` for an item won at 5,000 ETB.
- **Actionable Remediation**:
  - Generate payment intents server-side, pulling amounts directly from database records.

1.11.5 8.5 Wallet Double-Spend / Negative Balance

- **Risk Profile**: Critical. Rapid concurrent requests spending the same balance twice.
- **Codebase Verification & Defenses**:
  - Stress-tested in `scripts/concurrency-security-stress.js` :
    - Distributed Redis lock (`guessx:bid-fee:${auctionId}:${userId}:lock`) prevents concurrent processing.
    - Database balance update uses atomic SQL decrements checking `wallet_balance >= :amount` .
- **Testing Methodology**:
  - Issue 25 concurrent fee deduction requests against an account with an exact balance of 10 ETB.
- **Actionable Remediation**:

- Combine distributed locking with database `CHECK (wallet_balance >= 0)` constraints.

1.11.6 8.6 Fee / Charge Bypass

- **Risk Profile**: Critical. Placing bids while skipping the required participation fee.
- **Codebase Verification & Defenses**:
  - `BiddingService.placeBid` enforces `service_fee_paid === true`.
  - For wallet payments, fee deduction and bid placement are bound through verified single-use session keys.
- **Testing Methodology**:
  - Call `POST /auctions/:id/bid` directly without executing the prior fee payment step.
- **Actionable Remediation**:
  - Enforce single-use payment verification tokens in Redis that are consumed atomically upon bid registration.

1.11.7 8.7 Workflow & State Machine Bypass

- **Risk Profile**: High. Executing workflow steps out of order (e.g., confirming payment for an auction not won).
- **Codebase Verification & Defenses**:
  - State machine checks require:
    - User must match `auction.winner_user_id`.
    - Auction must be in `status: CLOSED` and `payment_status: PENDING`.
    - If payment deadline lapses, status transitions to `PAYMENT_DEFAULTED` and payments are rejected.
- **Testing Methodology**:
  - Call `POST /payments/:auctionId/confirm` for an ACTIVE auction or an auction won by another participant.
- **Actionable Remediation**:
  - Validate all workflow prerequisites inside transactional boundaries.

1.11.8 8.8 Business Limit Bypass (Max Bids per User)

- **Risk Profile**: High. Exceeding the 150-bid limit per auction through parallel requests.
- **Codebase Verification & Defenses**:
  - Atomic Redis counter (`INCR guessx:auction:${id}:user:${userId}:bids`) tracks bid counts.
  - Rejects submissions immediately when count exceeds `MAX_BIDS_PER_USER_PER_AUCTION = 150`.

- **Testing Methodology**:
  - Submit 50 concurrent bids when an account already has 140 recorded bids.
- **Actionable Remediation**:
  - Check and increment counters atomically prior to accepting submissions.

1.11.9 8.9 Mass Assignment / Over-Posting

- **Risk Profile**: High. Smuggling administrative attributes into customer requests.
- **Codebase Verification & Defenses**:
  - Global `ValidationPipe` with `whitelist: true` and `forbidNonWhitelisted: true` strips and rejects non-schema properties.
- **Testing Methodology**:
  - Send `PATCH /auth/profile` with `{ "role": "admin", "wallet_balance": 50000 }`.
- **Actionable Remediation**:
  - Maintain explicit DTOs for every route and forbid dynamic payload binding to ORM entities.

1.11.10 8.10 Transaction Replay Attacks

- **Risk Profile**: High. Replaying captured bid requests to duplicate submissions without authorization.
- **Codebase Verification & Defenses**:
  - `NonceGuard` requires unique `x-bid-nonce` and `x-bid-timestamp` headers.
  - Nonces stored in Redis with 60-second TTL using atomic `SET NX`. Replayed nonces are rejected immediately.
- **Testing Methodology**:
  - Capture a valid bid request and resend it verbatim 500ms later.
- **Actionable Remediation**:
  - Require cryptographic client nonces and enforce a 30-second clock skew window.

1.11.11 8.11 Currency & Rounding Manipulation

- **Risk Profile**: Medium. Exploiting fractional sub-cent amounts to create rounding discrepancies.
- **Codebase Verification & Defenses**:
  - All currency amounts are strictly normalized to 2 decimal places using `toFixed(2)` and stored in `DECIMAL(12, 2)` columns.
  - DTO validation enforces `maxDecimalPlaces: 2`.
- **Testing Methodology**:
  - Submit bid amounts with 3 to 6 decimal places (`12.3456`, `1.001`).

- **Actionable Remediation**:
  - Normalize all monetary values to 2 decimal places prior to processing.

1.11.12 8.12 Privileged Business Function Abuse (Administrator Bidding)

- **Risk Profile**: Critical. Administrators participating in auctions, exploiting insider knowledge, or manipulating results.
- **Codebase Verification & Defenses**:
  - **Administrator Bidding Restriction Rule**:
    - Strictly enforced at bid initiation across `PaymentController` (fee link creation, wallet fee payment) and `BiddingController`.
    - If `req.user.role === 'admin'`, the action is blocked immediately with HTTP 403 Forbidden: *"Administrators are not allowed to participate in bidding."*
    - Prevents administrators from navigating through payment flows only to be rejected downstream.
  - **Conflict of Interest Guard**:
    - Administrative force-close verifies that the acting administrator has no historical bids on the target auction.
- **Testing Methodology**:
  - Authenticate with administrative credentials and attempt to create a bid fee link, deduct a fee from a wallet, or submit a bid value.
- **Actionable Remediation**:
  - Maintain strict separation of duties, rejecting admin participation across all bidding gateways, and logging all privileged overrides to immutable audit trails.

1.12 9. Automated Penetration Test Verification

The platform maintains automated regression and security test suites executed prior to release:

Test Script	Target Focus Area	Test Count	Pass Status
`scripts/e2e-security-flow.js`	Full lifecycle auth, fail-closed JWT, SSRF URL blocking, CSRF guards, webhook HMAC verification, and admin bidding rejection.	24 Scenarios	**24 / 24 Passing (100%)**

Test Script	Target Focus Area	Test Count	Pass Status
`scripts/concurrency-security-stress.js`	High-concurrency race testing, wallet double-spend prevention, distributed Redis mutexes, fee bounds (`@Min(1.00)`, `@Max(10000)`), and nonces.	25 Scenarios	**25 / 25 Passing (100%)**
`auction-engine/test/security-controls.spec.ts`	Unit-level verification of role gates, SSRF private IP blocking, replay guards, and exception filters.	13 Suites	**Passing**

1.13 10. Remediation & Hardening Roadmap

Priority	Control Area	Hardening Action	Implementation Status
P0	**Admin Bidding Prohibition**	Strict separation of duties enforced at bid and fee initiation.	**Implemented**
P0	**Concurrency & Double-Spend**	Distributed Redis mutex locks and atomic database updates.	**Implemented**
P0	**Replay Protection**	Nonce and timestamp validation with Redis TTL.	**Implemented**
P1	**SSRF Defense**	Private IP blocking, HTTPS enforcement,	**Implemented**

Priority	Control Area	Hardening Action	Implementation Status
		and MIME validation in `ImageService`.	
P1	**Error Sanitization**	`GlobalExceptionFilter` masking stack traces and 5xx details from clients.	**Implemented**
P1	**Fee Bounds Validation**	Enforce `@Min(1.00)` and `@Max(10000)` on all fee deduction endpoints.	**Implemented**
P2	**Centralized Log Offloading**	Forward immutable audit logs to write-once WORM cloud storage.	Recommended for Production
P2	**mTLS Mesh**	Implement mutual TLS for container-to-container service calls.	Recommended for Production